

RI402

Dizajn kompajlera

dr.sc. Edin Pjanić

Pregled predavanja

- Implementacija regularnih izraza
- Konačni automati

Kompajler

- Leksička analiza
- Parsiranje
- Semantička analiza
- Generisanje koda

Leksička analiza: dizajn

- Korak 1:
 - Definirati konačan skup tokena (zavisi od jezika...)
- Korak 2:
 - Za svaku klasu tokena opisati koji stringovi im pripadaju

Leksička analiza: implementacija

- Implementacija mora :
 - Prepoznati podstringove koji korespondiraju sa tokenima
 - Leksem: sekvenca karaktera koji odgovaraju uzorku za neki token (konkretna instanca tokena)
 - Identifikovati klasu tokena za svaki leksem
<klasa tokena, leksem>
 - Obično se odbace tokeni koji nisu od interesa za parser
 - Prazna mjesta, komentari

Regularni izrazi

Specifikacija regularnih jezika.

- Načini zapisivanja:
 - $AA^* \quad \Leftrightarrow A^+$
 - $A + B \quad \Leftrightarrow A \mid B$
 - $A + \varepsilon \quad \Leftrightarrow A?$
 - $'c' + 'x' + 'W' \quad \Leftrightarrow [cxW]$
 - $'a' + 'b' + 'c' + \dots + 'z' \quad \Leftrightarrow [a-z]$
 - *Komplement od* $[ARa-fgi] \quad \Leftrightarrow [^ARa-fgi]$
- Specifikacija elemenata jezika
 - $s \in L(R)$
- Problem: izdvajanje leksema (tokena) iz sekvence znakova

Kako se vrši
prepoznavanje, izdvajanje i
klasifikacija
leksema (tokena)?

Prepoznavanje i izdvajanje leksema (tokena)

1. Napisati regexp za lekseme svake klase tokena:

- Number = digit^+
- Keyword = "if" + "then" + "else" + "while" + ...
- Identifier = letter (letter + digit)*
- Whitespace = (' ' + '\t' + '\n')⁺
- Assign = '='
- ...

Prepoznavanje i izdvajanje leksema (tokena)

2. Konstruisati regexp R čiji $L(R)$ će sadržavati sve lekseme za sve tokene:

- $R = \text{Keyword} + \text{Number} + \text{Identifier} + \dots$
- $R = R_1 + R_2 + R_3 + \dots$
 - $L(R) = L(R_1) \cup L(R_2) \cup L(R_2) \cup \dots$
 - $L(R) = \bigcup_{j=1,2,3,\dots} R_j$

Prepoznavanje i izdvajanje leksema (tokena)

3. Neka je ulaz sekvenca znakova $x_1, x_2, x_3, \dots, x_n$:
 - Za $1 \leq i \leq n$ provjeravamo da li $x_1 \dots x_i \in L(R)$
4. Ako jeste, onda je sigurno
 - $x_1 \dots x_i \in L(R_j)$ za neko j u R_j sa prethodnog slajda
5. Uklanjammo prepoznatu sekvencu $x_1 \dots x_i$ iz ulaza i idemo na korak 3.
 - Uzimamo naredni prefiks sa ulaza...

U slučaju da imamo više poklapanja...

S1: Koliku sekvencu znakova (prefiks) sa ulaza uzimamo?

- $x_1...x_i \in L(R)$
- $x_1...x_j \in L(R), i \neq j$ Npr. $x_1...x_i...x_j$
- *Uzima se najduža sekvenca*

Primjeri:
if
if123

S2: Koji token odabrati u slučaju više poklapanja iste dužine?

- $x_1...x_i \in L(R_j)$
- $x_1...x_i \in L(R_k)$
- Prioritet (redoslijed u specifikaciji)

S3: Slučaj nepoklapanja ni sa jednim pravilom?

- Ovo ne treba dozvoliti
- Definirati posebno pravilo za poklapanje sa bilo kojim stringom i postaviti ga na kraju specifikacije (error msg)

Regularni izrazi u leksičkoj analizi

Elegantna i precizna specifikacija uzoraka stringova (klasa tokena).

Potrebni su dodaci za rješavanje slučajeva koji se ne mogu riješiti samim regularnim izrazima (npr. prethodni slajd).

Neke karakteristike dobrih algoritama za leksičku analizu:

- jedan prolazak kroz ulazni string
- relativno mali broj operacija po karakteru

Pri tome želimo što bolju algoritamsku složenost.

Konačni automat (automat sa konačnim brojem stanja)

Jedan od načina implementacije regularnih izraza.

Engleski naziv:

- Finite automaton (FA)
- Finite state machine (FSM)

Mašina konačnog stanja, automat konačnog stanja, konačni automat, ili jednostavno automat, je vrsta matematičkog kompjucionog modela (ulaz → izlaz).

To je apstraktna mašina koja može biti u tačno jednom od konačnog broja **stanja** u bilo kojem trenutku.

FSM može prelaziti iz jednog stanja u drugo kao odgovor na neke ulaze. Promjena iz jednog stanja u drugo naziva se **prelaz** ili **tranzicija**.

Konačni automat (FA) je jednostavna idealizirana mašina koja se koristi za prepoznavanje uzoraka unutar ulaza koji se sastoji od nekog skupa znakova (ili nekog drugog alfabeta).

Posao FA je da prihvati ili odbije ulaz u zavisnosti od toga da li se uzorak definisan pomoću FA pojavljuje u ulazu.

Konačni automat (automat sa konačnim brojem stanja)

Konačni automat je definisan petorkom (Σ, S, s, F, T) :

- Ulaznim alfabetom (karakter, ulazni signal) Σ
- Konačnim skupom stanja automata S
- Početnim stanjem s
- Skupom tzv. prihvatajućih stanja F (F je podskup od S)
- Skupom prelaza (tranzicija) T

Terminologija

Prelaz (tranzicija):

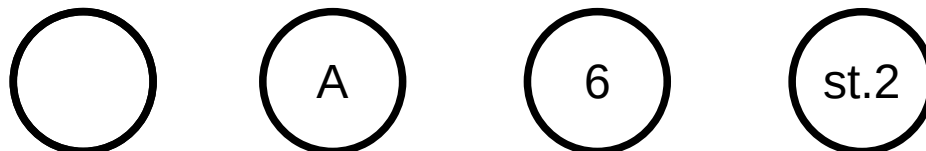
- Oznaka: $S_1 \rightarrow^a S_2$
- Značenje: Ako je automat u stanju s_1 , za ulaz a automat prelazi u stanje s_2

Prihvatajuće stanje, u slučaju regularnih izraza:

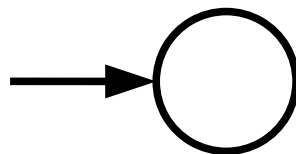
- Samo ako je **kraj ulaza** a automat je u **prihvatajućem** stanju, tada automat **prihvata** ulaz ($\Rightarrow$ ispravan ulaz).
- U suprotnom, automat **odbija** ulaz ($\Rightarrow$ neispravan ulaz). Primjeri slučajeva kad FSM odbija ulaz:
 - Došli smo do kraja ulaza a nije prihvatajuće stanje.
 - Automat nema definisan prelaz za dati ulaz

Grafički zapis (prikaz, definicija) automata

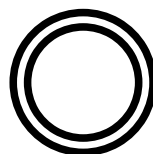
- Stanje



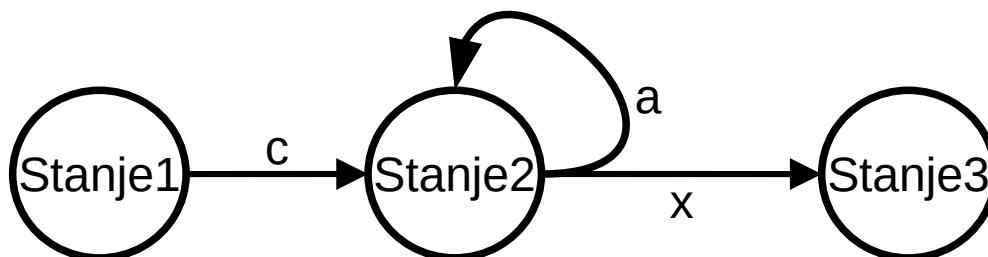
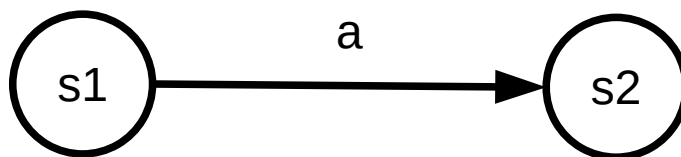
- Početno stanje



- Prihvatajuće stanje

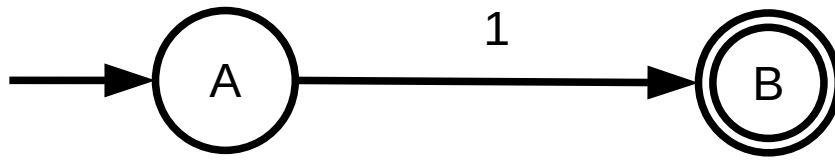


- Prelaz (tranzicija)



Primjer

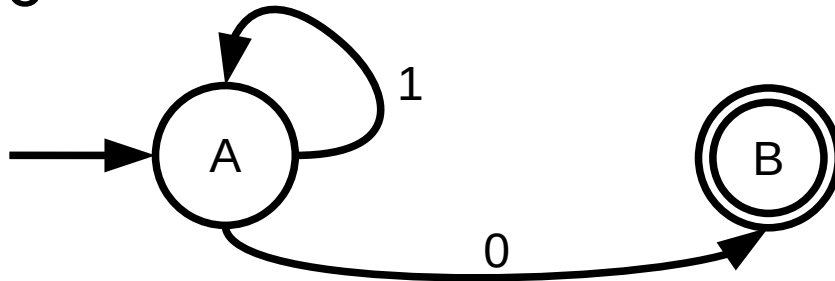
- Automat koji prihvata samo “1”



- Automat ne prihvata: “0”, “10”, “11”, itd.
- Uopšteno: Jezik nekog konačnog automata je skup svih stringova koje taj automat prihvata.

Primjer

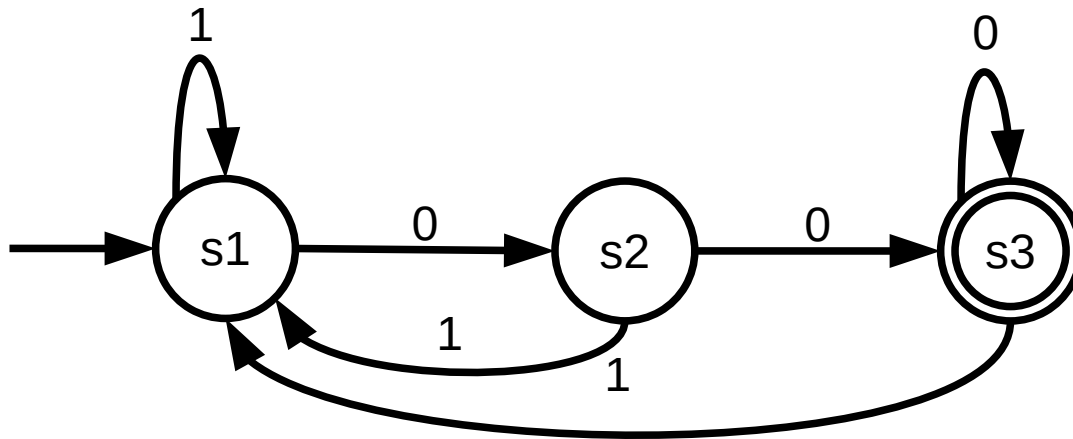
- Automat koji prihvata bilo koji broj jedinica “1” iza kojih slijedi “0”



Stanje	Ulaz	Novo stanje
A	0	B
A	1	A

Tabela tranzicija (prelaza)

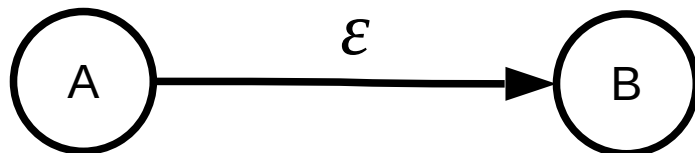
- Koji jezik prepoznaje naredni automat:



Stanje	Ulaz	Novo stanje
s1	0	s2
s1	1	s1
s2	0	s3
s2	1	s1
s3	0	s3
s3	1	s1

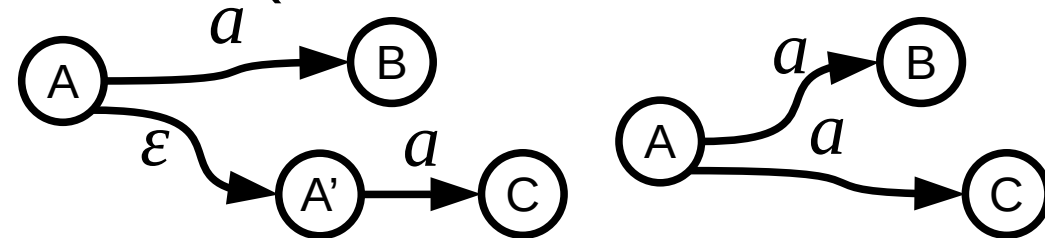
Pojam: ϵ - prelaz

- Prelaz za koji se ne konzumira ulaz, tj. automat može samostalno izvršiti ovaj prelaz

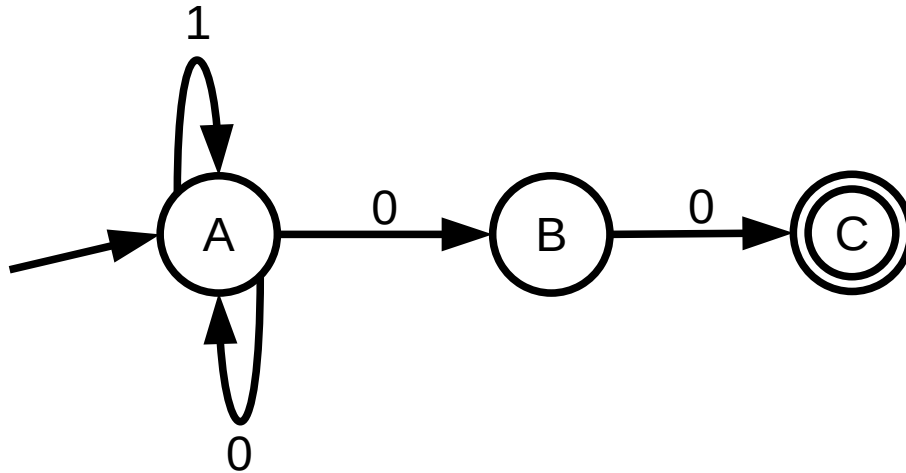


Vrste konačnih automata

- Deterministički konačni automati (*deterministic finite automata* – *DFA*)
 - Ne postoje ε -prelazi
 - Najviše jedan prelaz po ulaznom znaku u svakom stanju
 - Ima samo jednu prihvatajuću putanju za dati ulazni string
- Nedeterministički konačni automati (*nondeterministic finite automata* – *NFA*)
 - Mogu postojati ε -prelazi
 - Mogu sadržavati više prelaza po jednom ulaznom znaku za dato stanje
 - Može postojati više prihvatajućih putanja za dati ulazni string
 - NFA prihvata ulazni string ako bilo koja putanja vodi do prihvatajućeg stanja



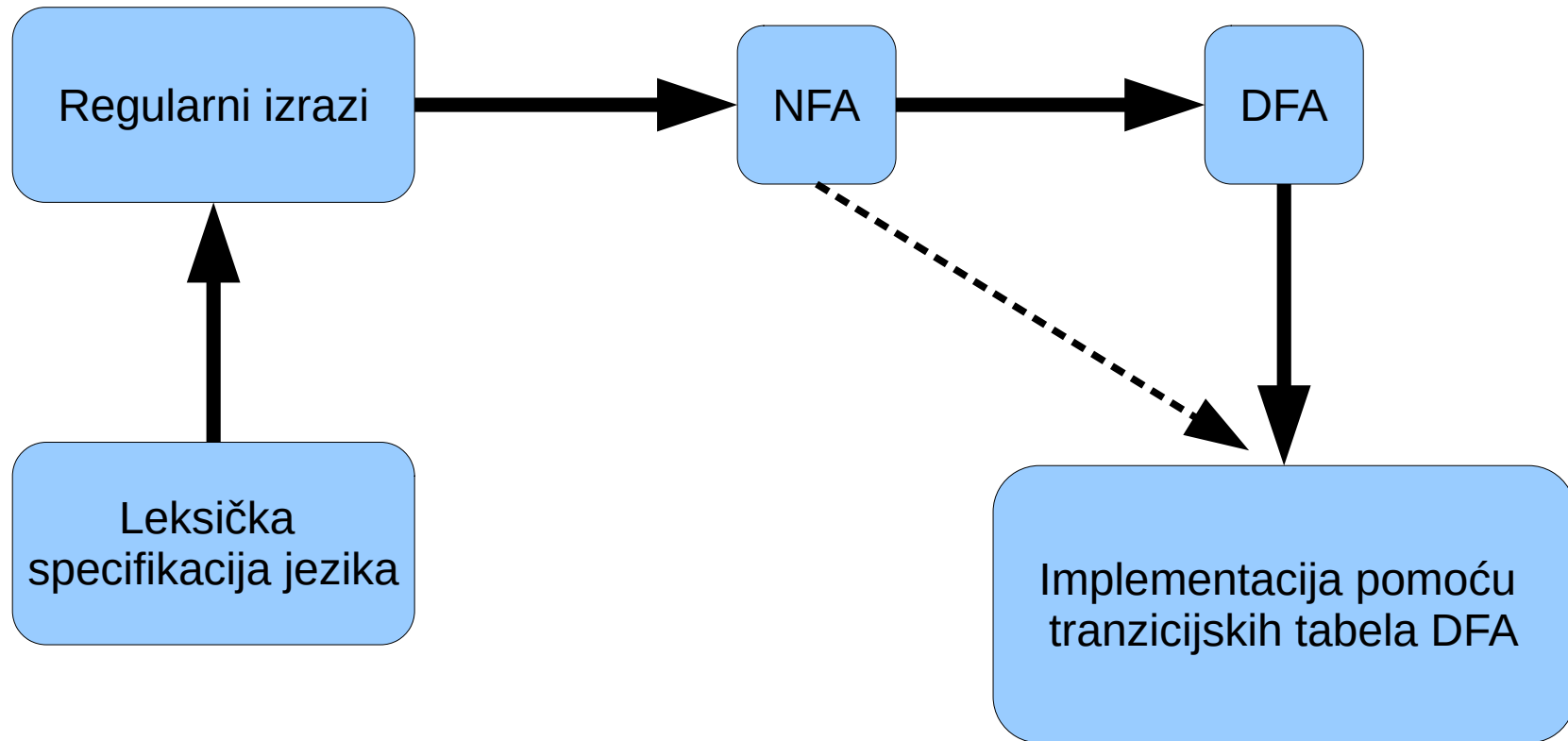
Nedeterministički automat – primjer



- Da li prihvata ulaz: 1 0 0
- Stanja nakon datog ulaza: {A} {A, B} {A, B, C}

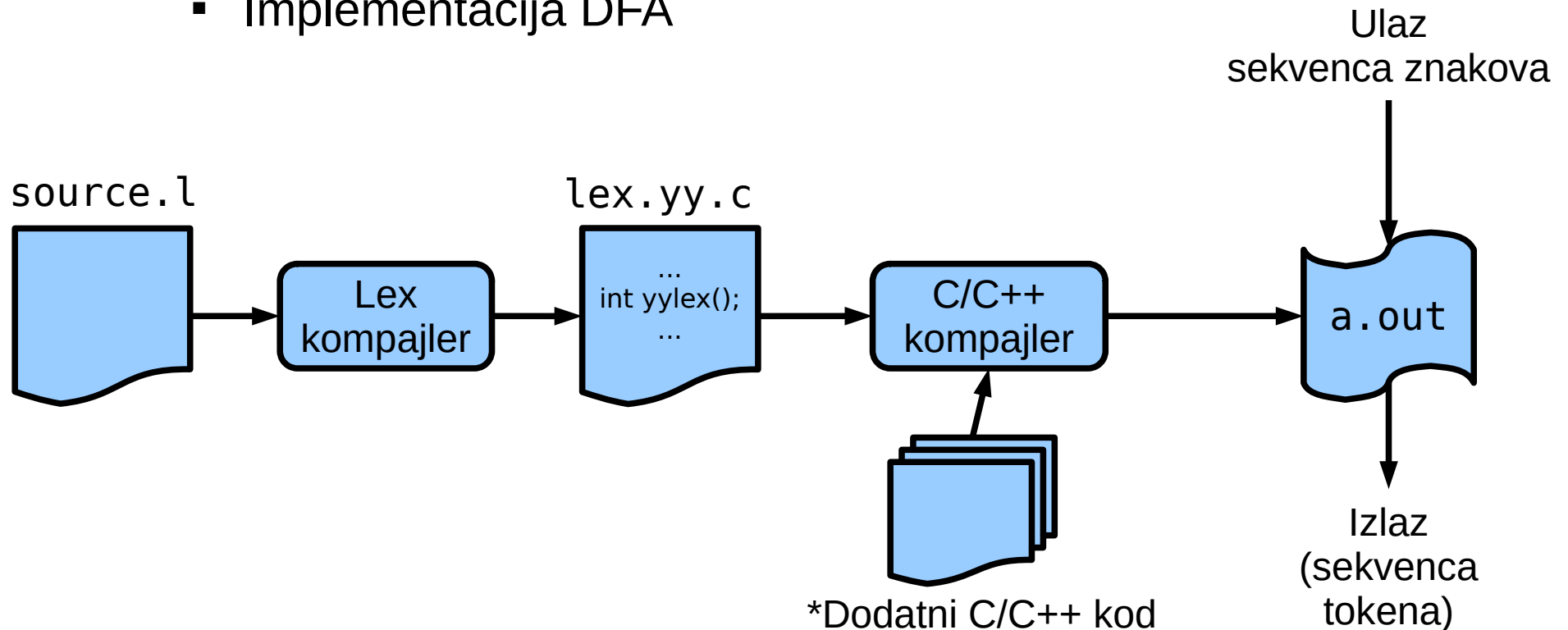
PRIHVATA

Stanje u kojem je NFA u datom trenutku zapisuje se kao skup mogućih stanja. NFA može biti u bilo kojem od tih stanja.



Lex (flex)

- Alat za generisanje leksičkog analizatora
- Regularni izrazi: opis uzoraka za tokene
 - Transformišu se u automate sa konačnim brojem stanja (DFA)
- Lex kompajler proizvodi `lex.yy.c` fajl
 - Implementacija DFA



Struktura Lex programa

Sastoji se od tri sekcije rastavljene oznakom:

%%

%{

C deklaracije/definicije
(ovaj dio se kopira u lex.yy.c)

%}

Regex definicije

Deklaracije

%%

Leksička pravila

%%

Pomoćne C funkcije

Definisanje:

- uzoraka za lekseme/tokene (regex) i
- akcija (C kod) koje će se izvršiti u slučaju pronalaska uzoraka u ulaznoj sekvenci znakova.

Akcije idu u funkciju **yylex()**.

Format leksičkih pravila:

uzorak { akcija }

Direktno se kopira u lex.yy.c

Primitivi za definisanje uzorka

Metakarakter	Odgovara stringu
.	bilo koji karakter osim nove linije
\n	nova linija
*	Klinijeva iter.: nula ili više kopija prethodnog izraza
+	Klinijeva +: jedan ili više kopija prethodnog izraza
?	opcija: nula ili jedna kopija prethodnog izraza
^	početak linije
\$	kraj linije
a b	a ili b
(ab) +	jedna ili više kopija grupe ab
"a+b"	doslovan izrazu naveden u navodnicima
[abc]	jedan od karaktera navedenih u uglastim zagradama (a ili b ili c)

Izraz	Odgovara stringu
abc	
abc*	
abc+	
a(bc)+	
a(bc)?	
[abc]	
[a-z]	
[a\ -z]	
[-az]	
[A-Za-z0-9]+	
[\t\n]+	
[^ab]	
[a^b]	
[a b]	
a b	

Procesiranje ulaza

- Funkcija za procesiranje ulaza
 - `int yylex(void)` – procesira ulaz i izvršava akcije sve dok ne dođe do kraja ulaza (vraća 0) ili je u akciji naredba `return`. Obično ova funkcija vraća klasu tokena (o tome kasnije).
- Koraci pri procesiranju:
 1. Analizira se ulaz i traži se string koji odgovara nekom uzorku (najduži pa po prioritetu).
 2. Za pronađeni uzorak, leksem se postavlja dostupnim preko `ytext` a dužina leksema preko `yleng`.
 3. Izvršava se specificirana akcija (ako je zadana).

Pri tome se mogu koristiti **globalne** Lex varijable:

- `ytext` - pointer na početak leksema, NULL terminirani C string (tip `char *`)
- `yleng` - dužina pronađenog leksema (tip `int`)
- `ylval` – može se koristiti za vrijednost asociranu sa tokenom (defaultno tip `int`)

Primjer – broj znakova, riječi i linija

```
%{  
    int numChars = 0, numWords = 0, numLines = 0;  
}%  
  
%%  
  
\n        {numLines++; numChars++;}  
[^ \t\n]+ {numWords++; numChars += yyleng;}  
.  
        {numChars++;}  
  
%%  
  
int main() {  
    yylex();  
    printf("%d\t%d\t%d\n", numChars, numWords, numLines);  
}
```

Kompajliranje i izvršavanje

```
$ flex brojzrl.l  
$ gcc -o brojanje lex.yy.c -ll  
$ ./brojanje < brojzrl.l  
$ 258 34 16
```

gcc – C kompajler
Moguće je koristiti bilo koji C
kompajler, kao npr. clang.

Lex biblioteka: uključuje funkcije yywrap() i
main(), ako ih mi ne definišemo.
O ovome ćemo dodatno diskutovati na
nekom od narednih predavanja.

Broj znakova, riječi i redova u fajlu brojzrl.l